

Start coding or generate with AI.

Assumptions for Simulated Cannabis E-Commerce Dataset (100 rows)

1. Product Attributes

- Product ID: Unique identifier (e.g., "p001", "p002", ...)
- Product Name:
 - Varies by strain + format + quantity (e.g., "OG Kush 1/8 oz", "Blue Dream Vape Cartridge 500mg")
 - Includes common cannabis strain names (OG Kush, Blue Dream, Girl Scout Cookies, etc.)
 - Product types: flower, vape, edible, concentrate
 - Naming inconsistencies (e.g., "OG Kush 1/8 oz" vs. "OG Kush Eighth", or typos)
- Category:
 - "Hybrid Flower", "Sativa Flower", "Indica Flower", "Vape Cartridge", "Edible", "Concentrate"
- Vendor Name: Simulate ~10 different vendors with realistic names
- THC %: Range from ~15% to 30% for flowers, or potency in mg for vapes/edibles (e.g., 500mg THC)
- CBD %: Range from 0% to 5% typically
- Price: Based on product type and quantity, e.g., 20–60 for 1/8 oz flower, 30–60 for vape cartridges
- Availability: Boolean or inventory count (e.g., 0-100 units)
- Description: Short text with strain effects, flavor notes, or product benefits (semi-randomized)
- Tags: List of keywords (e.g., "relaxing", "uplifting", "citrus", "pain relief")
- Date Added: Recent date within last 6 months

2. Data Inconsistencies / Noise

- Include misspellings or alternate product names
- Random missing values for THC% or CBD% (to simulate real messy data)
- Some products have incomplete descriptions
- Vendor names may vary slightly (e.g., "GreenLeaf Co." vs "Green Leaf Co.")
- Categories sometimes mislabeled or abbreviated ("Sativa", "Indica", "Hybrid", "Vape", etc.)

3. Relational Structure

- Products linked to vendors and categories
- Multiple products from same vendor with slight variations
- Some duplicate or near-duplicate products (to test similarity matching)

```
# import libraries
import random
import numpy as np
import pandas as pd
from datetime import datetime, timedelta
```

```
# set random seeds for reproducibility
random.seed(42)
np.random.seed(42)
```

```
# lists that hold common strains, product categories, vendors, and tags that
# reflect a realistic cannabis e-commerce environment.
```

```
strains = [
    "OG Kush", "Blue Dream", "Girl Scout Cookies", "Sour Diesel", "Maui Wowie",
    "Wedding Cake", "Pineapple Express", "Purple Urkle", "Green Crack",
    "Jack Herer", "Lemon Cherry", "Ice Cream Cake", "CA Gold Rush"]
```

```
strain_types = ["Indica", "Hybrid", "Sativa"]
```

```
categories = ["Flower", "Vape Cartridge", "Edible", "Concentrate"]
```

```
tags_by_strain = {
    "Indica": ["relaxing", "sleepy", "pain relief",
               "calming", "sedative", "body high"],
    "Sativa": ["uplifting", "energetic", "creative",
               "focused", "euphoric", "cerebral"],
    "Hybrid": ["relaxing", "sleepy", "pain relief", "calming",
               "uplifting", "energetic", "creative", "focused", "euphoric"] }
```

```
vendors = [
    "GreenLeaf Co.", "Herbal Haven", "CannaWorld", "Bud Masters",
    "Elevate Extracts", "Pure Bloom", "Zen Garden", "Happy Trails",
    "Emerald Fields", "Sunset Wellness" ]
```

```
# create fct to assign tags based on strain type choose 2 to 5 tags
```

```
def assign_tags(strain_type):
    possible_tags = tags_by_strain[strain_type]
    n_tags = random.randint(2, 5)
    return random.sample(possible_tags, k = n_tags)
```

```
# create fct to simulate thc level based on category and strain type
```

```
def simulate_thc(category, strain_type):
    thc_ranges = {
        "Flower": (15, 30),
        "Vape Cartridge": (50, 90),
        "Edible": (5, 20),
        "Concentrate": (70, 90) }
```

```

low, high = thc_ranges.get(category, (10, 25)) # if no category uses 10, 25
if strain_type == "Indica":
    high = max(low, high - 5) # indicas slightly lower thc
elif strain_type == "Sativa":
    high = min(100, high + 5) # sativas higher but no more than 100
# hybrid no change
return round(random.uniform(low, high), 1)

```

```
# create fct to assign potency mg based on category
```

```

def random_potency_mg(category):
    if category in ["Vape Cartridge", "Concentrate"]:
        return random.choice([250, 500, 750, 1000])
    elif category == "Edible":
        return random.choice([10, 25, 50, 100])
    else:
        return None

```

```
# create fct to generate random cbd %, mostly low values
```

```

def random_cbd():
    if random.random() < 0.8:
        return round(random.uniform(0, 2), 1)
    else:
        return round(random.uniform(2, 5), 1)

```

```

# generate prod names variants with some noisy variations
# create name variants with with alt spellings, lowercase, hyphens
# strain qty listed twice to increase chance of selection

```

```

def random_product_name(strain, category, qty):
    name_variants = [
        f"{strain} {qty}",
        f"{strain} {qty}",
        f"{strain} {qty}".replace("1/8 oz", "Eighth"),
        f"{strain} {qty}".replace("500mg", "Half Gram"),
        f"{strain} {qty}".replace("1000mg", "One Gram"),
        f"{strain} - {qty}",
        f"{strain} {qty}".lower() if random.random() < 0.5 else f"{strain} {qty}",
        None
    ]

    # filter out None values and pick randomly
    name_variants = [n for n in name_variants if n is not None]
    return random.choice(name_variants)

```

```
# create fct to assign quantity string based on category
```

```

def assign_quantity(category):
    if category == "Flower":
        return random.choice(["1/8 oz", "3.5g", "7g"])
    elif category == "Vape Cartridge" or category == "Concentrate":
        return random.choice(["500mg", "1000mg"])
    elif category == "Edible":
        return random.choice(["10 pieces", "20 pieces", "50 pieces",
                               "100g", "200g"])
    else:
        return None

```

```
# creat fct to est price based on category and quantity with noise
```

```

def random_price(category, qty):
    base_prices = {
        "Flower": 40,
        "Vape Cartridge": 45,
        "Edible": 25,
        "Concentrate": 50
    }

    price = base_prices.get(category, 40)
    if qty:
        if "g" in qty or "oz" in qty:
            if "1/8" in qty or "eighth" in qty.lower():
                price *= 1
            elif "3.5" in qty:
                price *= 1.5
            elif "7" in qty:
                price *= 2.5
        elif "mg" in qty:
            try:
                mg = int(''.join(filter(str.isdigit, qty)))
                price = price * mg / 500
            except:
                pass

    # add random variation +/- 10%

    price = price * random.uniform(0.9, 1.1)
    return round(price, 2)

```

```

# create fct to generate random availability with some out of stock cases
# if the number generator is less than 0.05, return 0
# if the number generator is less than 0.10 but greater than 0.05, return a number from 1 to 99
# else, return a number from 100 to 1000

```

```

def random_availability():
    rand_num = random.random()
    if rand_num < 0.05:
        return 0
    elif rand_num < 0.20:
        return random.randint(1, 99)
    else:
        return random.randint(100, 1000)

```

```
# create fct to generate a random date in last 6 months
```

```
def random_date():
```

```
start = datetime.today() - timedelta(days=180)
return start + timedelta(days=random.randint(0, 180))
```

```
# initialize list to hold product rows

rows = []
```

```
# generate 100 product rows

for i in range(1, 101):
    # pick a category randomly
    category = random.choice(categories)

    # pick a strain randomly
    strain = random.choice(strains)

    # assign strain type randomly, but realistically you could map strains to types
    strain_type = random.choice(strain_types)

    # assign quantity string based on category
    qty = assign_quantity(category)

    # generate product name with noise
    name = random_product_name(strain, category, qty)

    # pick a vendor randomly
    vendor = random.choice(vendors)

    # simulate thc level based on category and strain type
    thc = simulate_thc(category, strain_type)

    # generate cbd %
    cbd = random_cbd() if random.random() > 0.05 else None

    # assign potency mg if applicable
    potency_mg = random_potency_mg(category)

    # estimate price based on category and quantity
    price = random_price(category, qty or (str(potency_mg) + "mg" if potency_mg else None))

    # assign tags based on strain type
    tags = assign_tags(strain_type)

    # availability count
    availability = random_availability()

    # date added field
    date_added = random_date().strftime('%Y-%m-%d')

    # optional description (simplified)
    description = f"{strain} {strain_type} {category.lower()} with {'', '.join(tags)})."
    if random.random() < 0.1:
        description = None # simulate missing descriptions

    # append dictionary for one product row
    rows.append({
        "product_id": f"p{i:03d}",
        "name": name,
        "category": category,
        "strain_type": strain_type,
        "strain": strain,
        "vendor": vendor,
        "THC_pct": thc,
        "CBD_pct": cbd,
        "potency_mg": potency_mg,
        "price_usd": price,
        "availability": availability,
        "description": description,
        "tags": tags,
        "date_added": date_added
    })
```

```
# convert to df

df = pd.DataFrame(rows)
```

```
# display first 10 rows

print(df.head(10))
```

product_id	name		category	strain_type	\		
0	p001	OG Kush 3.5g	Flower	Sativa			
1	p002	Sour Diesel 1000mg	Concentrate	Hybrid			
2	p003	Blue Dream One Gram	Concentrate	Sativa			
3	p004	Sour Diesel - 50 pieces	Edible	Sativa			
4	p005	Sour Diesel - 7g	Flower	Sativa			
5	p006	CA Gold Rush 1/8 oz	Flower	Indica			
6	p007	Blue Dream 100g	Edible	Hybrid			
7	p008	OG Kush 1000mg	Concentrate	Indica			
8	p009	Green Crack Half Gram	Vape Cartridge	Sativa			
9	p010	Wedding Cake - 7g	Flower	Indica			
	strain	vendor	THC_pct	CBD_pct	potency_mg	price_usd	\
0	OG Kush	Herbal Haven	28.5	0.8	NaN	54.36	
1	Sour Diesel	CannaWorld	84.0	1.9	750.0	46.02	
2	Blue Dream	Pure Bloom	84.4	0.5	750.0	54.85	
3	Sour Diesel	Herbal Haven	17.2	0.3	100.0	23.85	
4	Sour Diesel	Happy Trails	22.9	0.5	NaN	94.93	
5	CA Gold Rush	Zen Garden	21.0	0.9	NaN	38.01	
6	Blue Dream	Elevate Extracts	19.6	0.2	50.0	26.71	
7	OG Kush	Elevate Extracts	73.6	0.2	1000.0	53.16	
8	Green Crack	Pure Bloom	69.7	0.4	750.0	40.69	
9	Wedding Cake	Happy Trails	17.1	3.7	NaN	99.45	
	availability	description					\
0	833	OG Kush Sativa flower with energetic, euphoric...					
1	144	Sour Diesel Hybrid concentrate with sleepy, en...					
2	954	Blue Dream Sativa concentrate with uplifting, ...					
3	132	Sour Diesel Sativa edible with cerebral, creat...					
4	324	Sour Diesel Sativa flower with cerebral, eupho...					

```
5      649  CA Gold Rush Indica flower with body high, rel...
6      68   Blue Dream Hybrid edible with pain relief, ene...
7      993  OG Kush Indica concentrate with sleepy, calm...
8      81   Green Crack Sativa vape cartridge with euphor...
9      520  Wedding Cake Indica flower with calming, sleep...

                                tags  date_added
0                                [energetic, euphoric, uplifting] 2025-07-29
1  [sleepy, energetic, creative, pain relief, upl... 2025-06-10
2                                [uplifting, focused, creative] 2025-05-17
3                                [cerebral, creative, uplifting] 2025-05-04
4  [cerebral, euphoric, focused, creative] 2025-03-20
5                                [body high, relaxing] 2025-04-22
6  [pain relief, energetic, creative] 2025-02-13
7                                [sleepy, calming, body high] 2025-07-18
8                                [euphoric, energetic, uplifting] 2025-02-28
9  [calming, sleepy, relaxing, body high, sedative] 2025-06-12

# convert df to csv

df.to_csv('cannabis-prod-data.csv', index = False)

# download the file

from google.colab import files

files.download('cannabis-prod-data.csv')
```